

RISC-V LINUX 嵌入式编程技术 · 第一章

开发环境篇

RISC-V 平台认知 · 开发环境安装 · 镜像烧录与启动

— 1.1 / 1.2 / 1.3 —

目标平台：LicheePi 4A / TH1520

操作系统：RevyOS

工具链：rui + GNU Toolchain

三节递进： 从环境搭建到工程判断

1.1

RISC-V 平台认知 与 ruyi 环境安装

建立从 ISA 到应用的层次概念，安装 ruyi 包管理器与交叉编译工具链，完成首次工具链验证。

- 平台分层：ISA → SoC → 开发板 → OS → 工具
- ruyi 安装、软件源更新、工具链查询
- 编译器版本与目标三元组验证

1.2

镜像烧录、首次启动 与开发通道建立

将 RevyOS 镜像安全写入启动介质，完成首次启动与系统检查，建立串口 / SSH / SCP 开发通道。

- 镜像校验与安全烧录
- 首次启动检查（系统、网络、时间）
- SSH 密钥登录 + SCP 双向传输

1.3

ruyi device provision 适用边界

理解自动化设备准备的输入、输出与风险。对比手工流程与自动 provision，建立风险驱动的决策框架。

- provision 目的、能力与局限
- 手工 vs 自动：六维对比
- 六项风险检查与决策路径

从 ISA 到应用：课程平台的六个层次

RISC-V 是开放指令集架构（ISA），不是一款芯片或开发板。建立分层思维是后续排查「镜像不支持」「工具链不匹配」等问题的基础。

RISC-V ISA	开放指令集架构规范
SoC · TH1520	处理器核与片上系统
开发板	LicheePi 4A
操作系统	RevyOS（Linux 发行版）
开发工具	ruyi、GCC 工具链、SSH、串口
课程应用	实验代码与综合项目



ruyi：包管理入口，不是编译器

ruyi 是 RuyiSDK 的包管理与环境管理入口。它负责发现和获取工具链——真正编译代码的是 GCC / binutils。两者的关系类似「应用商店」与「应用程序」。

ruyi 的四个核心能力

- 软件源与包信息 — 查询可用工具链、镜像与设备支持
- 工具链获取 — 下载并安装 RISC-V GNU Toolchain
- 环境与配置管理 — 管理不同项目的工具链环境
- 设备支持查询 — 查看受支持的开发板与 provision 方式

安装后第一步永远是 ruyi --version 和 ruyi --help——本机帮助输出是最可靠的命令入口。

```
# 安装验证

$ command -v ruyi

/home/user/.local/bin/ruyi


$ ruyi --version

ruyi x.y.z


# 更新软件源

$ ruyi update


# 查询工具链

$ ruyi list | grep toolchain


# 验证编译器

$ riscv64-unknown-linux-gnu-gcc

-dumpmachine

riscv64-unknown-linux-gnu
```

环境准备与安装验证清单

硬件准备

- **主机** — Linux x86_64（推荐），macOS / Windows 备选
- **目标板** — LicheePi 4A（TH1520 SoC）
- **供电** — 稳定电源适配器（核对板卡规格）
- **USB 数据线** — 确认支持数据传输，非仅充电线
- **USB-TTL 串口模块** — 3.3V 电平
- **网络** — 网线或 Wi-Fi，可访问 RuyiSDK 软件源

⚠ 常见陷阱

- 部分 USB-C 线只能供电不能传数据 → 无法识别设备时先换线
- 串口电平必须为 3.3V → 5V 可能损坏板卡
- 工具链需选 linux-gnu 而非 unknown-elf（裸机）

软件检查

- **ruyi 安装** — 官方渠道获取，不从不明来源复制
- **PATH 检查** — `command -v ruyi` 需返回有效路径
- **工具链前缀** — 确认 `riscv64-unknown-linux-gnu-`
- **sysroot 定位** — `-print-sysroot` 或用 `-v` 输出查找
- **基础工具** — `curl/wget`、`sha256sum`、`file`

最小验证程序

```
int main(void) { return 0; }  
# 编译后 file 输出应含 RISC-V ELF
```

不要用主机 `./a.out` 验证 → 架构不同，必报错。正确验收 = `file` 确认目标架构。

本节成果：建立平台认知，工具链就绪

01 平台分层图 能准确区分 ISA、SoC、开发板、OS、工具、应用六个层次。	02 接口标注 能识别 LicheePi 4A 的供电、串口、网络 and 启动介质位置。
03 环境清单 硬件、线缆、主机软件均已确认状态（已有 / 缺少 / 待确认）。	04 ruyi 环境 ruyi 已安装，软件源可更新，工具链已查询并记录版本与三元组。
05 编译器验证 目标三元组含 riscv64 + linux-gnu；最小程序 file 确认 RISC-V ELF。	06 实验日志 实验 1.1 准备材料与完整终端日志齐备。

关键认知：「LicheePi 4A 是 RISC-V」不准确。正确表述：LicheePi 4A 是基于 RISC-V 架构（ISA）、搭载 TH1520 SoC 的开发板，运行 RevyOS 操作系统。

镜像烧录不是「复制文件」

烧录工具将引导程序、内核、设备树和根文件系统按分区结构写入目标介质。错误的镜像或错误的目标设备 = 无法启动，严重时覆盖主机磁盘。

- 1

校验镜像

sha256sum 与发布值比对，不一致 = 停止烧录
- 2

识别目标设备

一次只连一块待烧录板，用烧录工具的设备列表命令确认
- 3

执行烧录

严格按该镜像版本的发布说明操作，不拼接其他板卡/年份的命令
- 4

等待完成

工具明确报告成功后再断电。首次启动可能因文件系统扩展耗时较长

[镜像 → 引导程序 + 内核 + 设备树 + 根文件系统 → eMMC/SD]

首次启动警示

不要因短暂无输出立即断电——首次启动可能进行文件系统扩展、密钥生成或初始化。优先使用串口观察引导日志，屏幕无画面 ≠ 未启动。

三条通道，三个角色：串口 · SSH · SCP



串口（UART）

本地调试通道 · 最后保障线

- 不依赖网络，直接观察启动日志
- 适合修复网络配置、诊断启动失败
- 接线：GND-GND、TX → RX、RX → TX
- 波特率按板卡资料设定
- ⚠ 3.3V 电平，禁用 5V 电源脚



SSH

远程主工作通道 · 日常开发入口

- 加密网络终端，适合日常开发
- 首次连接须**核对主机指纹**
- 推荐 Ed25519 密钥登录替代密码
- 登录后验证 hostname + uname -m
- 前提：网络可达 + sshd 运行



SCP

文件传输通道 · 部署流水线

- 复用 SSH 连接传输文件
- 上传后板端 sha256sum 校验
- 下载板端文件到主机分析
- 传到用户有写权限的目录
- 双向传输，两端哈希一致 = 成功

黄金组合：串口留着备用（断网时的救生索），SSH 做主通道，SCP 负责搬文件。

首次启动：系统检查四步走

启动后立即确认的五件事

1 系统版本 — `cat /etc/os-release` → 确认 RevyOS

2 架构 — `uname -m` → `riscv64`

3 账户 — `id` → 记录当前用户

4 主机名 — `hostnamectl` → 记录与管理

5 磁盘 — `df -h` → 确认存储空间

网络与时间

6 IP 地址 — `ip -br addr`

7 路由 — `ip route` → 确认默认网关

8 时间 — `timedatectl` → 时间错误会导致 TLS 和 apt 失败

apt 软件源检查

```
# 查看当前软件源
$ grep -R --no-filename -h '^deb ' \
  /etc/apt/sources.list \
  /etc/apt/sources.list.d 2>/dev/null

# 更新（先确认链路/地址/路由/时间正常）
$ sudo apt update
```

apt update 失败 ≠ 需要立即换源。

先排查 DNS → 时间 → 网络 → 仓库可用性。

系统时间严重错误时，TLS 证书和仓库元数据校验都会失败。

SSH 服务启用：

sudo systemctl enable --now ssh

SSH 首次连接与密钥登录

从密码到密钥：三步建立安全通道

- 1

首次连接 — 核对指纹

ssh user@board-ip → 看到主机指纹时**必须通过串口或受信任渠道核对**，不要习惯性输入 yes
- 2

生成密钥对

ssh-keygen -t ed25519 -C "riscv-course"

私钥留在主机，绝不复制给他人
- 3

部署公钥

ssh-copy-id user@board-ip

写入板端 ~/.ssh/authorized_keys

SCP 双向传输

- ↑↓

上传 + 下载 + 校验

主机 sha256sum → scp 上传 → 板端 sha256sum 对比
板端文件 scp 下载到主机分析
两端哈希一致 = 传输无损坏

```
# 生成 Ed25519 密钥
$ ssh-keygen -t ed25519 -C "riscv-course"

# 部署公钥到板端
$ ssh-copy-id user@board-ip

# 免密验证
$ ssh user@board-ip 'hostname; uname -m'

# SCP 双向传输 + 校验
$ echo 'test' > transfer-test.txt
$ sha256sum transfer-test.txt
$ scp transfer-test.txt user@board-ip:/tmp/
$ ssh user@board-ip 'sha256sum /tmp/transfer-test.txt'
```

指纹变更警告：可能是开发板重装系统，也可能连错了设备。先核对 IP + 串口指纹，再清理 known_hosts 旧记录。

本节成果：板卡启动就绪，三通道全部打通

镜像校验记录

镜像来源、文件名、版本和 sha256 校验值完整保存。

烧录日志

完整烧录日志与首次启动记录——工具明确报告成功。

系统检查表

RevyOS、内核、用户、网络、时间和 apt 源均已核实。

串口通道

USB-TTL 3.3V 接线已确认，终端参数已记录——断网时最后的保障。

SSH 密钥登录

Ed25519 密钥对已部署，免密登录板端，hostname + uname -m 验证通过。

SCP 双向传输

上传下载均通过 sha256sum 校验，哈希一致。

故障排查口诀：「五类原因法」

开发板无法启动 → 拆解为 镜像 烧录 供电 启动介质 显示通道 五类，每类一个验证方法。串口优先于 SSH：没有 IP → SSH 连接超时 → 先用串口看 ip addr 和 sshd 状态。

provision 是什么：自动化设备准备

Device provision 是把设备准备成可使用状态的自动化过程——可能包括下载镜像、选择设备变体、写入存储介质或生成操作指引。



它做什么：将实验 1.2 中手工执行的多个步骤封装为一条命令，减少重复操作。

它不做什么：不会消除误选磁盘、断电或版本不匹配的风险。自动化只降低重复成本，不消除风险——「能运行命令」≠「适合运行命令」。

provision 的前提

- ① 设备在软件源中**明确列出**
- ② 变体与实物**一致**
- ③ 写入目标**唯一且可确认**
- ④ 重要数据**已备份**
- ⑤ 供电和数据连接**稳定**
- ⑥ 已准备串口或其他**恢复通道**

⚠ **关键认知：**自动化写入仍可能覆盖错误存储设备。目标不唯一时应断开无关设备，或改用 1.2 的手工流程。

手工流程 vs 自动化 provision：六维对比

手工流程（1.2） 每步可控，最可调试 控制粒度 每步可控，可随时停止 适用条件 只要有镜像和烧录工具 误操作风险 人为确认每一步，可发现异常 可调试性 高，每步可单独验证 重复效率 低，需重复执行相同命令 建议场景 首次使用、设备未列出、数据风险高	自动化 provision 高效，但需严格前置确认 控制粒度 流程封装，中断后需恢复 适用条件 设备必须在软件源中明确列出 误操作风险 自动化写入可能跳过确认 可调试性 低，失败后需拆解自动化步骤 重复效率 高，一条命令完成多步 建议场景 设备明确支持、重复部署、空白介质
---	--

手工优先

设备未在软件源中列出？ 名称相似但不完全匹配？ 存在重要数据？ → 走 1.2 手工流程。

自动适用

设备完全匹配 + 变体确认 + 新空白存储 + 六项风险检查全部通过 → 可考虑 provision。

六项风险检查 + 决策路径

在运行任何写入操作前必答

- ① 设备是否在当前软件源中明确列出？
- ② 变体是否与实物一致？
- ③ 写入目标是否唯一且可确认？
- ④ 重要数据是否已备份？
- ⑤ 供电和数据连接是否稳定？
- ⑥ 是否已准备串口或其他恢复通道？

查询 ruyi 帮助与设备列表

→ 设备与变体是否明确受支持？

是 → 核对六项 → 全部通过 → 执行 provision 并验证

否 → 使用手工流程（1.2），记录不支持原因

任一项不确认 → 停止自动写入

任一项不能确认，都应停止自动 provision。课程不要求在不安全条件下执行自动写入。

场景 A

设备完全匹配，但有重要数据

→ 手工流程（先备份，再操作）

场景 B

设备未列出但名称相似

→ 手工流程，记录「该设备在 ruyi 版本 X 中未列出」

场景 C

设备列出 + 新空白存储

→ 可考虑 provision（六项检查通过后）

核心原则：「提交不支持结论 + 手工替代方案」同样有效的实验结论。不要为完成实验强行执行 provision。

从搭建环境到建立工程判断力

1.1 平台认知与环境安装

- 掌握 ISA → SoC → 开发板 → OS → 工具的层次概念
- ruyi 包管理器安装与工具链查询
- 编译器版本与目标三元组验证
- 关键认知：RISC-V ≠ 操作系统 ≠ 开发板

1.2 镜像烧录与开发通道

- 镜像校验 → 安全烧录 → 首次启动验证
- 串口（备份）+ SSH（主力）+ SCP（传输）三通道
- Ed25519 密钥登录，SCP 双向传输 + 哈希校验
- 关键认知：屏幕无画面 ≠ 未启动

1.3 provision 适用边界

- provision = 自动化设备准备，不消除风险
- 六维对比：手工 vs 自动的优劣场景
- 六项风险检查驱动的决策框架
- 关键认知：不强行 provision 也是有效结论

↓ 第二章 ↓

第二章预告：工具链与工程 — host/target/sysroot 概念 → 交叉编译 Hello World → ELF 二进制分析 → Makefile 工程化 → 统一目录规范。环境已就绪，开始写第一行 RISC-V 代码。